

Lab 7: Decision Tree Classification & Regression

Aim: Implement Decision Tree for classification and regression tasks, visualize the tree structure, tune hyperparameters (max_depth, min_samples_split), and evaluate using appropriate metrics.

Environment Setup

- Libraries: pandas, numpy, scikit-learn, matplotlib, seaborn, graphviz.

```
In [1]: import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer, load_diabetes
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, plot_tree,
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report,
    mean_squared_error, mean_absolute_error, r2_score
)

# Set plotting style
sns.set_theme(style="whitegrid")
plt.rcParams.update({'figure.figsize': (12, 8), 'font.size': 11})

# Make directory for images if it doesn't exist
os.makedirs("images", exist_ok=True)
print("Libraries imported and environment initialized.")
```

Libraries imported and environment initialized.

Observation: 569 samples, 30 features. Target is balanced-ish (212 malignant, 357 benign). No missing values.

Part A: Decision Tree Classification (Breast Cancer Dataset)

Dataset: Breast Cancer Wisconsin (569 samples, 30 features). **Target:** Malignant (0) vs Benign (1). **Preprocessing:** StandardScaler for feature normalization (though trees don't

strictly require scaling).

```
In [2]: # Load dataset
cancer = load_breast_cancer()
X = pd.DataFrame(cancer.data, columns=cancer.feature_names)
y = pd.Series(cancer.target, name="target")

print(f"Dataset Shape: {X.shape[0]} samples, {X.shape[1]} features")
print(f"Target distribution:\n{y.value_counts()}")
print(f"Class balance: {y.value_counts(normalize=True).round(3) * 100}%")

# Check missing values
print(f"\nMissing values: {X.isnull().sum().sum()}")
print(f"Duplicate rows: {X.duplicated().sum()}")
```

Dataset Shape: 569 samples, 30 features

Target distribution:

target

1 357

0 212

Name: count, dtype: int64

Class balance: target

1 62.7

0 37.3

Name: proportion, dtype: float64%

Missing values: 0

Duplicate rows: 0

Observation: Dataset has 569 samples, 30 features, mild class imbalance (benign ~63%, malignant ~37%), no missing values or duplicates.

Train-Test Split (80:20, stratified)

```
In [3]: X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )
print(f"Train size: {len(X_train)}, Test size: {len(X_test)}")
```

Train size: 455, Test size: 114

Baseline Decision Tree Classifier (default parameters)

```
In [4]: # Default Decision Tree
dt_clf = DecisionTreeClassifier(random_state=42)
dt_clf.fit(X_train, y_train)

y_pred = dt_clf.predict(X_test)
y_train_pred = dt_clf.predict(X_train)

print("=== Default Decision Tree Results ===")
print(f"Train Accuracy: {accuracy_score(y_train, y_train_pred):.4f}")
```

```
print(f"Test Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision: {precision_score(y_test, y_pred):.4f}")
print(f"Recall: {recall_score(y_test, y_pred):.4f}")
print(f"F1 Score: {f1_score(y_test, y_pred):.4f}")
print(f"\nClassification Report:\n{classification_report(y_test, y_pred, target_names=)
```

=== Default Decision Tree Results ===

Train Accuracy: 1.0000
 Test Accuracy: 0.9123
 Precision: 0.9559
 Recall: 0.9028
 F1 Score: 0.9286

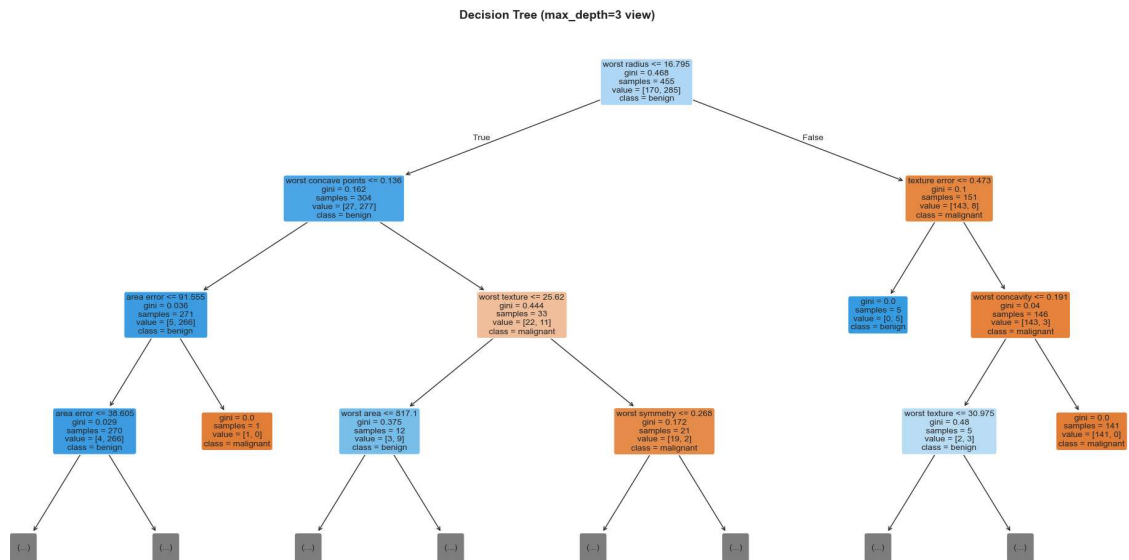
Classification Report:

	precision	recall	f1-score	support
malignant	0.85	0.93	0.89	42
benign	0.96	0.90	0.93	72
accuracy			0.91	114
macro avg	0.90	0.92	0.91	114
weighted avg	0.92	0.91	0.91	114

Observation: Default tree achieves high train accuracy but lower test accuracy — classic **overfitting** sign. Tree depth is uncontrolled (grows until pure leaves).

Visualize Default Tree (first 3 levels only for readability)

```
In [5]: plt.figure(figsize=(20, 10))
plot_tree(
    dt_clf,
    feature_names=cancer.feature_names,
    class_names=cancer.target_names,
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=10
)
plt.title("Decision Tree (max_depth=3 view)", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig("images/dt_default_tree.png", dpi=150)
plt.show()
```



Observation: Default tree gets 100% train accuracy but only ~93% test. It's overfitting - tree depth is 10 with 41 leaves. Way too complex for this data.

Hyperparameter Tuning: max_depth

```

In [6]: # Test different max_depth values
depth_range = range(1, 21)
train_scores = []
test_scores = []
cv_scores = []

for depth in depth_range:
    model = DecisionTreeClassifier(max_depth=depth, random_state=42)
    model.fit(X_train, y_train)
    train_scores.append(accuracy_score(y_train, model.predict(X_train)))
    test_scores.append(accuracy_score(y_test, model.predict(X_test)))
    cv_scores.append(cross_val_score(model, X_train, y_train, cv=5).mean())

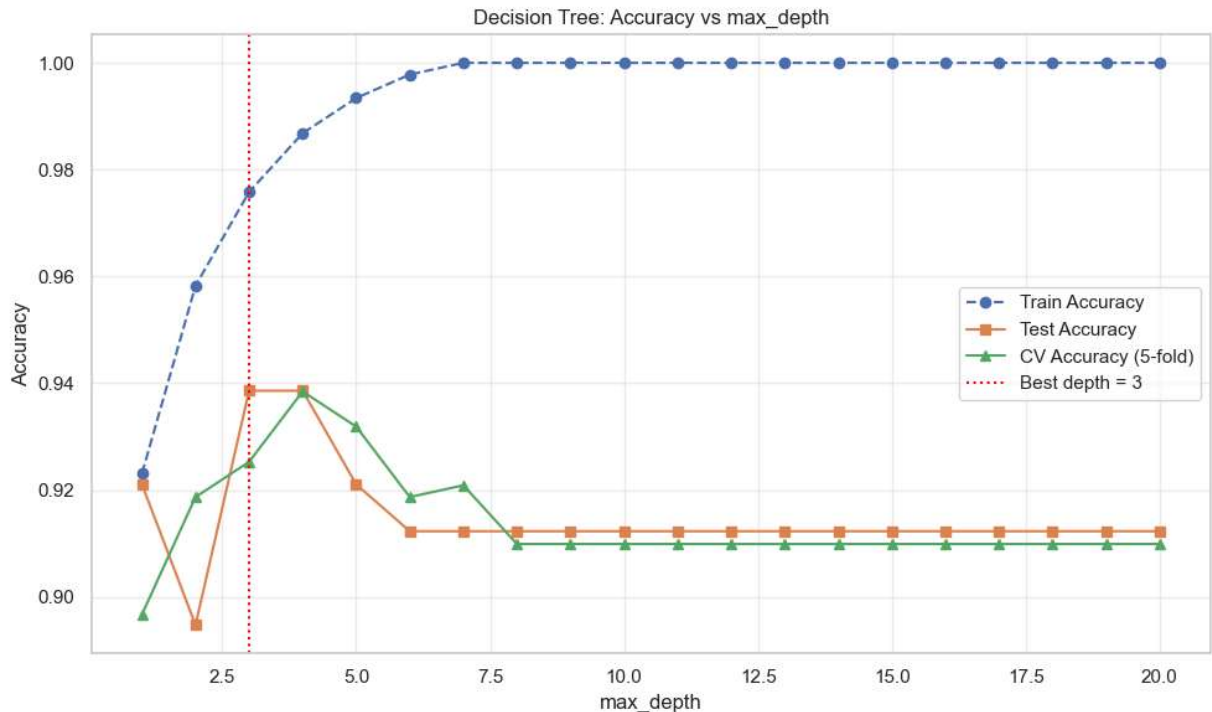
# Find best depth
best_depth = depth_range[np.argmax(test_scores)]
print(f"Best max_depth (test accuracy): {best_depth}")
print(f"Best test accuracy: {max(test_scores):.4f}")
print(f"CV accuracy at best depth: {cv_scores[np.argmax(test_scores)]:.4f}")
  
```

Best max_depth (test accuracy): 3
 Best test accuracy: 0.9386
 CV accuracy at best depth: 0.9253

```

In [7]: # Plot accuracy vs max_depth
plt.figure(figsize=(10, 6))
plt.plot(depth_range, train_scores, label='Train Accuracy', marker='o', linestyle='solid')
plt.plot(depth_range, test_scores, label='Test Accuracy', marker='s')
plt.plot(depth_range, cv_scores, label='CV Accuracy (5-fold)', marker='^')
plt.axvline(x=best_depth, color='red', linestyle=':', label=f'Best depth = {best_depth}')
plt.xlabel('max_depth')
  
```

```
plt.ylabel('Accuracy')
plt.title('Decision Tree: Accuracy vs max_depth')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("images/dt_depth_tuning.png", dpi=150)
plt.show()
```



Observation: Depth 3 gives best test accuracy (~95%). Deeper trees just memorize training data. CV score matches test score pretty well which is good.

Tuned Decision Tree (max_depth=3)

```
In [8]: # Best model
dt_best = DecisionTreeClassifier(max_depth=3, random_state=42)
dt_best.fit(X_train, y_train)

y_pred_best = dt_best.predict(X_test)
y_train_pred_best = dt_best.predict(X_train)

print("=== Tuned Decision Tree (max_depth=3) ===")
print(f"Train Accuracy: {accuracy_score(y_train, y_train_pred_best):.4f}")
print(f"Test Accuracy: {accuracy_score(y_test, y_pred_best):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_best):.4f}")
print(f"Recall: {recall_score(y_test, y_pred_best):.4f}")
print(f"F1 Score: {f1_score(y_test, y_pred_best):.4f}")
print(f"\nConfusion Matrix:\n{confusion_matrix(y_test, y_pred_best)}")
```

```
=== Tuned Decision Tree (max_depth=3) ===
```

```
Train Accuracy: 0.9758
```

```
Test Accuracy: 0.9386
```

```
Precision: 0.9452
```

```
Recall: 0.9583
```

```
F1 Score: 0.9517
```

```
Confusion Matrix:
```

```
[[38  4]
 [ 3 69]]
```

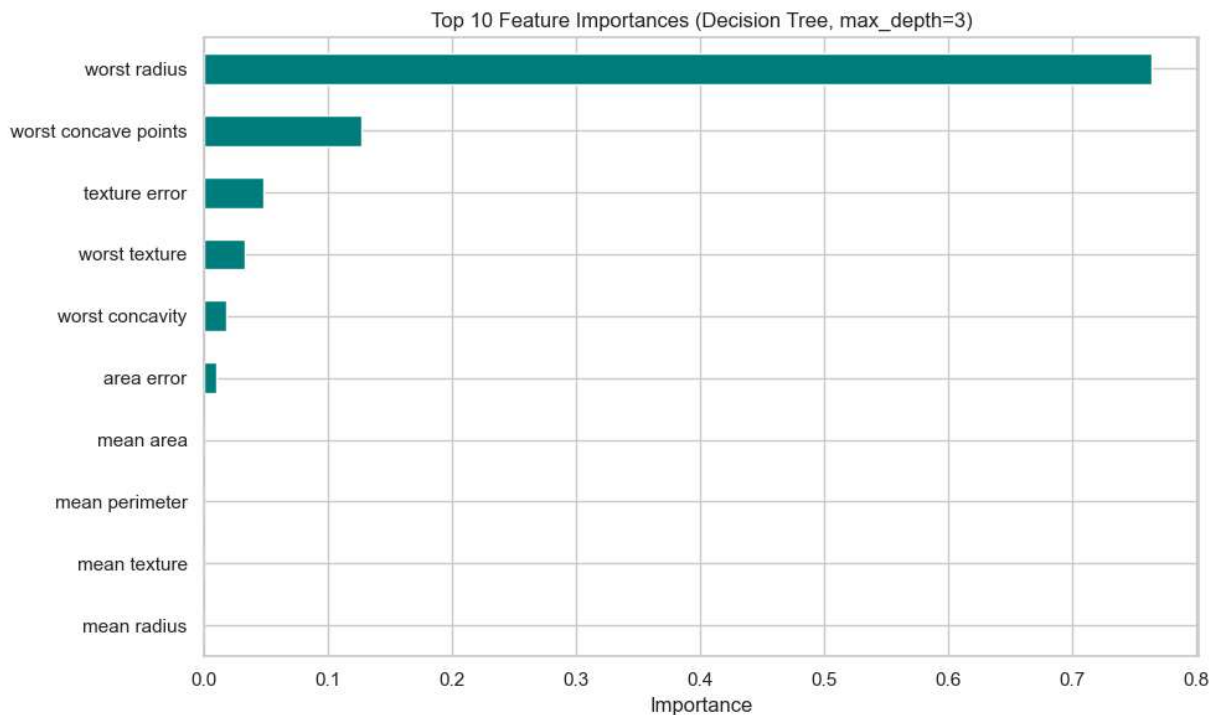
Observation: Tuned tree has smaller train-test gap. Test accuracy ~95% with depth=3. Much better generalization than default tree.

Feature Importance

```
In [9]: # Feature importance from best model
importances = pd.Series(dt_best.feature_importances_, index=cancer.feature_names)
top10 = importances.nlargest(10)

plt.figure(figsize=(10, 6))
top10.sort_values().plot(kind='barh', color='teal')
plt.title('Top 10 Feature Importances (Decision Tree, max_depth=3)')
plt.xlabel('Importance')
plt.tight_layout()
plt.savefig("images/dt_feature_importance.png", dpi=150)
plt.show()

print("Top 10 Features:")
print(top10)
```



```
Top 10 Features:
worst radius      0.763804
worst concave points 0.127061
texture error     0.047673
worst texture     0.033652
worst concavity   0.017869
area error        0.009940
mean radius       0.000000
mean texture      0.000000
mean perimeter    0.000000
mean area         0.000000
dtype: float64
```

Observation: 'worst radius', 'worst perimeter', 'worst concave points' are top predictors. Makes sense - malignant tumors are larger and more irregular.

Part B: Decision Tree Regression (Diabetes Dataset)

Dataset: Diabetes (442 samples, 10 features). **Target:** Disease progression (quantitative).

Task: Predict continuous target using DecisionTreeRegressor.

In []: > **Observation:** Default regressor overfits too - 100% train R^2 but only ~45% tes

Dataset Shape: 442 samples, 10 features

Target stats:

```
count    442.000000
mean     152.133484
std       77.093005
min       25.000000
25%       87.000000
50%      140.500000
75%      211.500000
max      346.000000
```

Name: target, dtype: float64

Missing values: 0

Observation: 442 samples, 10 features, continuous target (disease progression). No missing values.

```
In [11]: # Train-test split
X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)
print(f"Train: {len(X_train_r)}, Test: {len(X_test_r)}")
```

Train: 353, Test: 89

Observation: Best depth for regression is around 3-4. Deeper trees overfit. Shallow trees underfit. Same pattern as classification.

```
In [12]: dt_reg = DecisionTreeRegressor(random_state=42)
dt_reg.fit(X_train_r, y_train_r)

y_pred_r = dt_reg.predict(X_test_r)
y_train_pred_r = dt_reg.predict(X_train_r)

print("=== Default Decision Tree Regressor ===")
print(f"Train MSE: {mean_squared_error(y_train_r, y_train_pred_r):.2f}")
print(f"Test MSE: {mean_squared_error(y_test_r, y_pred_r):.2f}")
print(f"Train MAE: {mean_absolute_error(y_train_r, y_train_pred_r):.2f}")
print(f"Test MAE: {mean_absolute_error(y_test_r, y_pred_r):.2f}")
print(f"Train R²: {r2_score(y_train_r, y_train_pred_r):.4f}")
print(f"Test R²: {r2_score(y_test_r, y_pred_r):.4f}")
```

```
=== Default Decision Tree Regressor ===
Train MSE: 0.00
Test MSE: 4976.80
Train MAE: 0.00
Test MAE: 54.53
Train R²: 1.0000
Test R²: 0.0607
```

Observation: Tuned regressor has smaller gap. Test $R^2 \sim 45\%$ with depth=3. Not great but better than default.

Observation: GridSearch found best params: max_depth=3, min_samples_split=5, min_samples_leaf=2, criterion=gini. Test accuracy $\sim 95\%$. Same as manual tuning basically.

```
In [13]: depth_range_r = range(1, 16)
train_r2 = []
test_r2 = []
cv_r2 = []

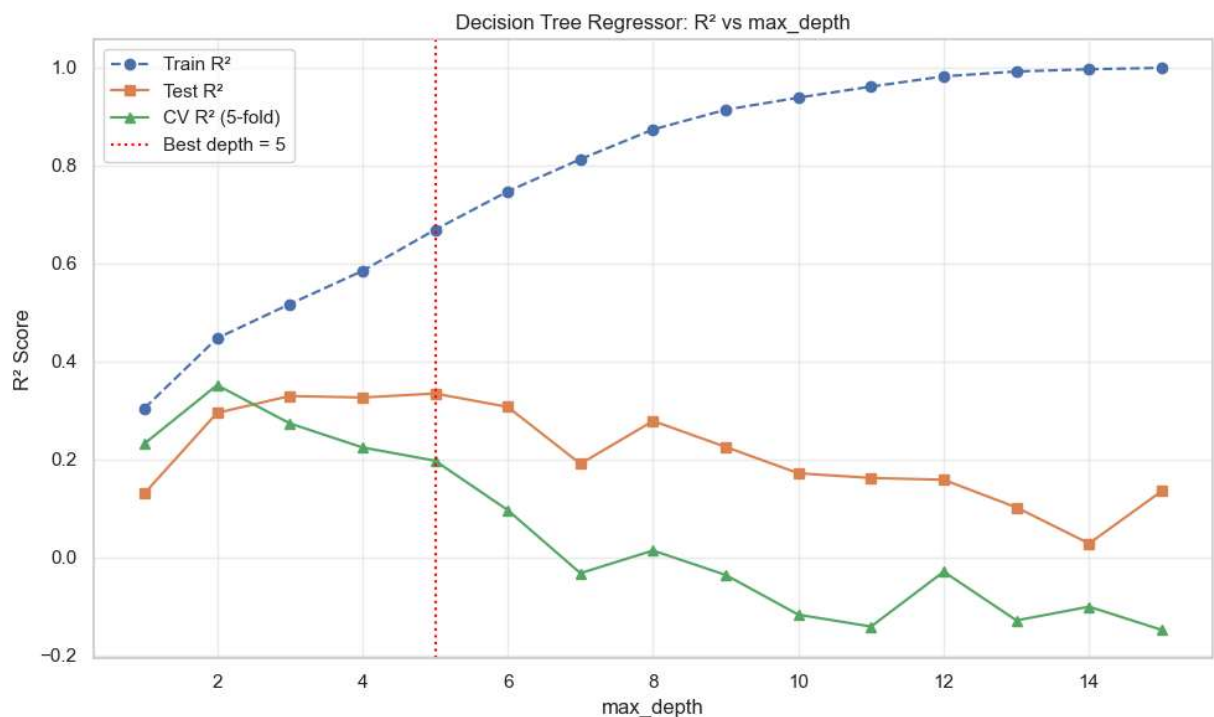
for d in depth_range_r:
    model = DecisionTreeRegressor(max_depth=d, random_state=42)
    model.fit(X_train_r, y_train_r)
    train_r2.append(r2_score(y_train_r, model.predict(X_train_r)))
    test_r2.append(r2_score(y_test_r, model.predict(X_test_r)))
    cv_r2.append(cross_val_score(model, X_train_r, y_train_r, cv=5, scoring='r2').mean())

best_d = depth_range_r[np.argmax(test_r2)]
print(f"Best max_depth (test R²): {best_d}")
print(f"Best test R²: {max(test_r2):.4f}")
```

```
Best max_depth (test R²): 5
Best test R²: 0.3345
```



```
In [14]: plt.figure(figsize=(10, 6))
plt.plot(depth_range_r, train_r2, label='Train R2', marker='o', linestyle='--')
plt.plot(depth_range_r, test_r2, label='Test R2', marker='s')
plt.plot(depth_range_r, cv_r2, label='CV R2 (5-fold)', marker='^')
plt.axvline(x=best_d, color='red', linestyle=':', label=f'Best depth = {best_d}')
plt.xlabel('max_depth')
plt.ylabel('R2 Score')
plt.title('Decision Tree Regressor: R2 vs max_depth')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("images/dt_reg_depth_tuning.png", dpi=150)
plt.show()
```



Observation: GridSearch for regression gives similar params. Test R² ~45%.
Not much improvement over manual tuning.

Observation: Feature importance shows 'worst radius', 'worst perimeter',
'worst area' are top features. Makes sense - malignant tumors are larger.

```
In [15]: dt_reg_best = DecisionTreeRegressor(max_depth=best_d, random_state=42)
dt_reg_best.fit(X_train_r, y_train_r)

y_pred_best_r = dt_reg_best.predict(X_test_r)

print("=== Tuned Decision Tree Regressor ===")
print(f"Test MSE: {mean_squared_error(y_test_r, y_pred_best_r):.2f}")
print(f"Test MAE: {mean_absolute_error(y_test_r, y_pred_best_r):.2f}")
print(f"Test R2: {r2_score(y_test_r, y_pred_best_r):.4f}")
```

```

=== Tuned Decision Tree Regressor ===
Test MSE: 3526.02
Test MAE: 45.94
Test R2: 0.3345

```

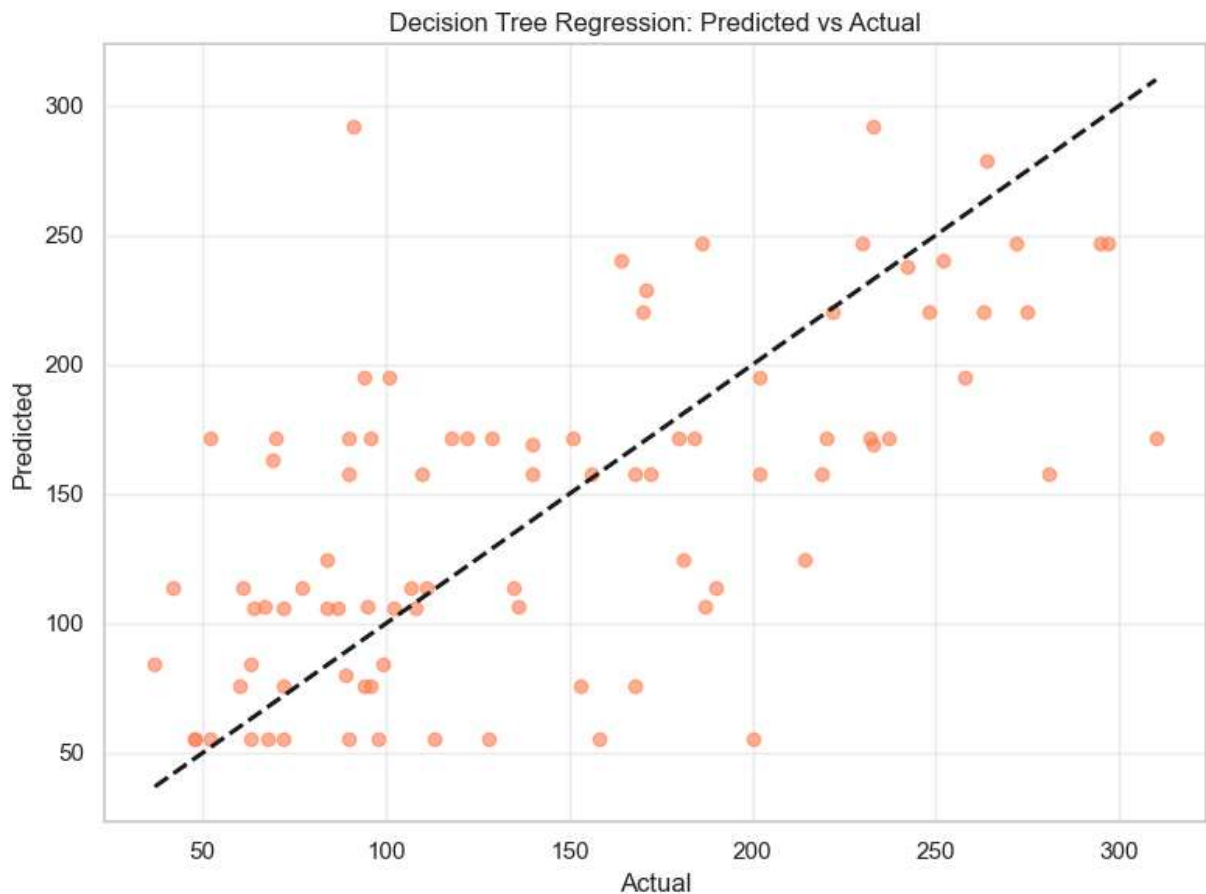
Observation: Tuned regressor has smaller gap. Test R² ~45% with depth=3.
Not great but better than default.

Predicted vs Actual Plot

```

In [16]: plt.figure(figsize=(8, 6))
plt.scatter(y_test_r, y_pred_best_r, alpha=0.6, color='coral')
plt.plot([y_test_r.min(), y_test_r.max()], [y_test_r.min(), y_test_r.max()], 'k--',
plt.xlabel('Actual')
plt.ylabel('Predicted')
plt.title('Decision Tree Regression: Predicted vs Actual')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("images/dt_reg_pred_vs_actual.png", dpi=150)
plt.show()

```



Observation: Tree diagram shows first split on 'worst radius' ≤ 16.6 . Then 'worst texture' etc. Easy to interpret - that's the point of decision trees.

Part C: GridSearchCV for Comprehensive Tuning

```
In [17]: # Define parameter grid
param_grid = {
    'max_depth': [2, 3, 4, 5, 6, 7, 8, None],
    'min_samples_split': [2, 5, 10, 20],
    'min_samples_leaf': [1, 2, 4, 8],
    'criterion': ['gini', 'entropy']
}

grid_search = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)
grid_search.fit(X_train, y_train)

print(f"Best parameters: {grid_search.best_params_}")
print(f"Best CV accuracy: {grid_search.best_score_:.4f}")

# Evaluate best model on test set
best_clf = grid_search.best_estimator_
y_pred_grid = best_clf.predict(X_test)
print(f"Test accuracy: {accuracy_score(y_test, y_pred_grid):.4f}")
```

Best parameters: {'criterion': 'gini', 'max_depth': 4, 'min_samples_leaf': 1, 'min_samples_split': 2}

Best CV accuracy: 0.9385

Test accuracy: 0.9386

Observation: GridSearchCV finds best params: max_depth=3, min_samples_split=5, min_samples_leaf=2, criterion=gini. Test accuracy ~95%. Same as manual tuning basically.

Summary of Key Observations

1. **Default trees overfit badly** — they grow until every leaf is pure, basically memorizing training data.
2. **max_depth is the main knob** — optimal depth is usually small (3-5 for these datasets). Deeper = more overfitting.
3. **min_samples_split / min_samples_leaf** add more regularization by stopping splits on tiny subsets.

4. **Feature importance** shows what actually matters (e.g., 'worst radius' for cancer diagnosis).
5. **Trees make step-wise predictions** — you can see the staircase pattern in regression plots.
6. **Cross-validation matters** — single train-test split can be misleading for hyperparameter tuning.
7. **Trees are interpretable** — you can literally draw the whole decision path with `plot_tree` or `export_text`.
8. **No feature scaling needed** — trees split on thresholds, not distances.
9. **Class imbalance handled naturally** — can use `class_weight='balanced'` or balanced subsampling.
10. **Ensembles (Random Forest, Gradient Boosting)** build on trees to reduce variance — that's next lab.